2º DAW · IES Amparo Sanz

Modelos de Arquitecturas Web y Protocolos de Comunicación

HTTP · Arquitecturas web · SSH



Curso 2026/2027

<indice/>

Índice

01

Protocolo HTTP

Cómo se comunican el navegador y el servidor: mensajes, métodos, códigos de estado y HTTPS.

02

Modelos de arquitecturas web

P2P, cliente-servidor y modelos con servidor de aplicaciones.

03

Protocolo SSH

Conexión remota y cifrada para administrar servidores.

01

Protocolo HTTP

HyperText Transfer Protocol: el idioma en el que hablan la web y los navegadores



<http/>

¿Qué es HTTP?

HTTP (HyperText Transfer Protocol) es el protocolo de la capa de aplicación que usan los navegadores y los servidores web para intercambiar información: páginas HTML, imágenes, CSS, JSON de una API, etc.

Cliente-servidor

El cliente envía una petición (request) y el servidor devuelve una respuesta (response).

Sin estado

Cada petición es independiente: el servidor no recuerda las anteriores.

Basado en texto

Hasta HTTP/1.1 los mensajes se pueden leer a simple vista.

Sobre TCP

Puerto 80 (HTTP) y 443 (HTTPS). HTTP/3 usa QUIC sobre UDP.

Si no tiene estado, ¿cómo recuerda al usuario?

Con cookies, sesiones o tokens (por ejemplo, JWT): así funcionan el login o el carrito.

<http/>

Anatomía de una URL

Cada recurso de la web se identifica con una URL. Así se descompone:



Protocolo

Cómo se comunican: http o https.

Dominio

Nombre del servidor. El DNS lo traduce a una IP.

Puerto

Entrada al servidor. Por defecto 80 o 443, por eso casi nunca se escribe.

Ruta

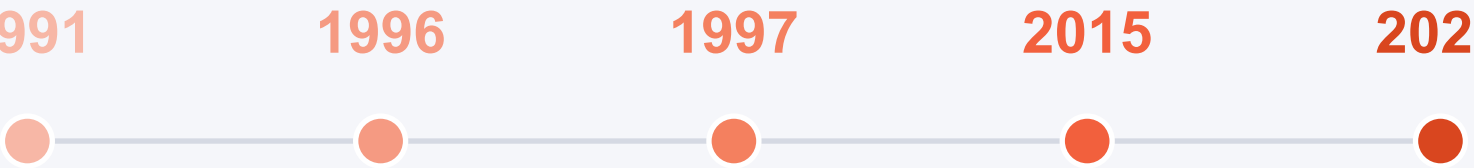
Dónde está el recurso dentro del servidor.

Parámetros

Datos extra con formato clave=valor después del ?

<http/>

Evolución de HTTP



HTTP/0.9

Solo método GET y solo HTML. No tenía cabeceras.

HTTP/1.0

Añade cabeceras, códigos de estado y otros tipos de contenido. Una conexión por petición.

HTTP/1.1

Conexiones persistentes (keep-alive), cabecera Host obligatoria y más métodos.

HTTP/2

Binario, multiplexación (varias peticiones por una conexión), compresión de cabeceras y server push.

HTTP/3

Usa QUIC sobre UDP en vez de TCP: más rápido y mejor en redes inestables.

<http/>

Petición y respuesta

PETICIÓN · cliente → servidor

```
GET /index.html HTTP/1.1
Host: www.ejemplo.com
User-Agent: Mozilla/5.0
Accept: text/html
(línea en blanco)
(sin cuerpo: un GET no envía datos)
```

RESPUESTA · servidor → cliente

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1250
(línea en blanco)
<!DOCTYPE html><html>...</html>
```

1. **Línea inicial** — Método + ruta + versión / versión + código
2. **Cabeceras** — Pares Nombre: valor
3. **Línea en blanco** — Separa cabeceras y cuerpo
4. **Cuerpo** — Datos o recurso (opcional)

<http/>

Métodos HTTP

		CRUD
GET	Obtener un recurso	Read
POST	Enviar datos o crear un recurso	Create
PUT	Reemplazar un recurso entero	Update
PATCH	Modificar parte de un recurso	Update
DELETE	Eliminar un recurso	Delete
HEAD	Como GET, pero solo cabeceras	—
OPTIONS	Métodos que permite (CORS)	—

Seguros

No modifican datos en el servidor.

GET HEAD OPTIONS

Idempotentes

Repetirlos varias veces da el mismo resultado. POST no lo es.

GET PUT DELETE

<http/>

Códigos de estado

Número de 3 cifras que indica cómo ha ido la petición. La primera cifra marca la categoría.

1xx

Informativo

100
Continue

101
Switching Protocols

2xx

Éxito

200
OK

201
Created

204
No Content

3xx

Redirección

301
Moved Permanently

302
Found

304
Not Modified

4xx

Error del cliente

400
Bad Request

401
Unauthorized

403
Forbidden

404
Not Found

5xx

Error del servidor

500
Internal Server Error

502
Bad Gateway

503
Service Unavailable

HTTPS: HTTP seguro



HTTP + TLS

Cifra la comunicación entre el cliente y el servidor. Hoy es obligatorio en la práctica.

`http://` puerto 80 · "No seguro"
`https://` puerto 443 · cifrado



Confidencialidad

Nadie puede leer los datos por el camino.



Integridad

Los datos no se pueden modificar sin que se detecte.



Autenticación

El servidor demuestra quién es con un certificado digital de una autoridad certificadora (p. ej., Let's Encrypt).

<http/>

¿Qué pasa al escribir una URL?

1

Escribes la URL

https://www.ejemplo.com en el navegador.

2

DNS

Traduce el dominio a una dirección IP.

3

Conexión

Conexión TCP y handshake TLS (por ser HTTPS).

4

Petición

El navegador envía GET / HTTP/1.1

5

Respuesta

El servidor responde 200 OK con el HTML.

6

Más peticiones

El navegador pide el CSS, el JS y las imágenes.

7

Renderizado

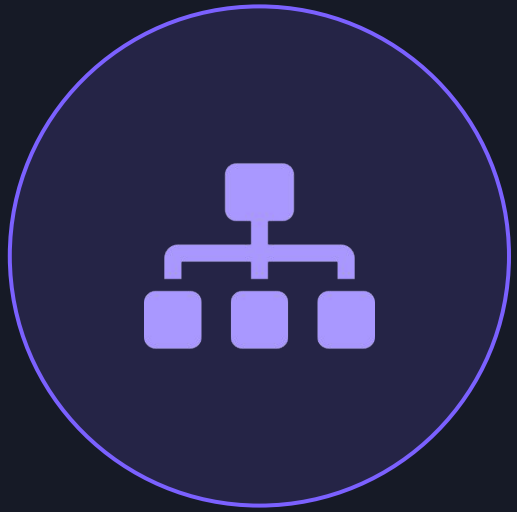
La página se muestra en pantalla.

HTTP conecta las capas de la arquitectura web. **SSH** sirve para administrar esos servidores.

02

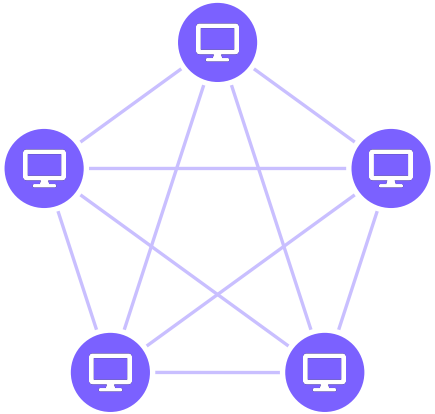
Modelos de arquitecturas web

Cómo se reparten el trabajo los equipos que forman una aplicación web



Modelo P2P (Peer to Peer)

Es una red descentralizada y distribuida en la que no existe un servidor central. Cada nodo (*peer*) actúa a la vez como cliente, al solicitar recursos, y como proveedor, al ofrecer los suyos propios: capacidad de procesamiento, almacenamiento y ancho de banda. Un ejemplo habitual es el intercambio de archivos mediante descargas compartidas o el streaming de audio y vídeo.



Ventajas

Escalabilidad: cuantos más nodos, mejor funciona la red

Robustez: no depende de un único servidor

Descentralización: ningún nodo es imprescindible

Costes repartidos: la carga no recae en un solo nodo

Desventajas

Falta de fiabilidad: no se garantiza que el recurso sea el deseado ni que no esté modificado

Difícil mantenimiento: actualizar un recurso obliga a hacerlo en todos los nodos

Modelo cliente-servidor

Reparte el trabajo entre servidores, que ofrecen un recurso o servicio, y clientes, que lo solicitan. Suelen comunicarse por red, aunque pueden estar en la misma máquina. El servidor web envía el recurso tal cual (HTML, imagen, sonido…), sin tratarlo.

1. El usuario pide una página con su URL
2. El servidor recibe la petición
3. Busca la página en su almacenamiento
4. Envía el código fuente por el mismo medio
5. El navegador lo interpreta y lo muestra

Ventajas

- Control centralizado de accesos, recursos y datos
- Clientes y servidores escalan por separado
- Se puede cambiar un servidor sin afectar a los clientes

Desventajas

- Saturación del sistema cuando muchos usuarios hacen peticiones al mismo tiempo
- Si el servidor cae, no se atienden peticiones
- Exige hardware y software específicos (más coste)

Modelos con servidor de aplicaciones

Un servidor web solo envía los archivos tal cual (recursos estáticos). Un servidor de aplicaciones ejecuta el código del recurso y devuelve el resultado a través del servidor web. Suele conectarse a una base de datos, en la misma máquina o en otro host de la red.



En la misma máquina

El servidor web recibe la petición. Si el recurso es dinámico, se lo pasa al servidor de aplicaciones, que lo ejecuta y devuelve el resultado para enviarlo al cliente.



Servidor de aplicaciones externo

El servidor web trabaja menos y necesita menos recursos. Un redirector manda a la aplicación solo lo que hay que ejecutar; lo estático no pasa.



Varios servidores de aplicaciones

Si la carga es alta, varios servidores comparten un servidor web. El redirector hace de balanceador y decide cuál ejecuta cada petición. Todos deben ser iguales.

03

Protocolo SSH

Secure Shell: controlar un servidor a distancia de forma segura y cifrada



<ssh/>

¿Qué es SSH?

SSH es un protocolo de red y un programa informático que permite a dos ordenadores comunicarse de forma segura y cifrada a través de una red.

Se usa sobre todo para conectarse a servidores remotos desde la línea de comandos:



Administrar sistemas



Ejecutar comandos



Transferir archivos

22

Puerto por defecto de SSH

Un concepto básico: es parecido a AnyDesk, pero desde la terminal y sin interfaz gráfica.

<ssh/>

Arquitectura cliente-servidor: el servidor

SERVIDOR SSH

Programa que se instala en el servidor para poder controlar a distancia

Desde la terminal

```
$ sudo apt install openssh-server
```

Este paso solo se hace una vez; una vez hecho esto, el servidor está listo para recibir conexiones.

EJEMPLO: OPENSSH,

es el paquete de software que incluye las herramientas
para conectarse como servicio para recibir conexiones

<ssh/>

Arquitectura cliente-servidor: el cliente

CLIENTE SSH

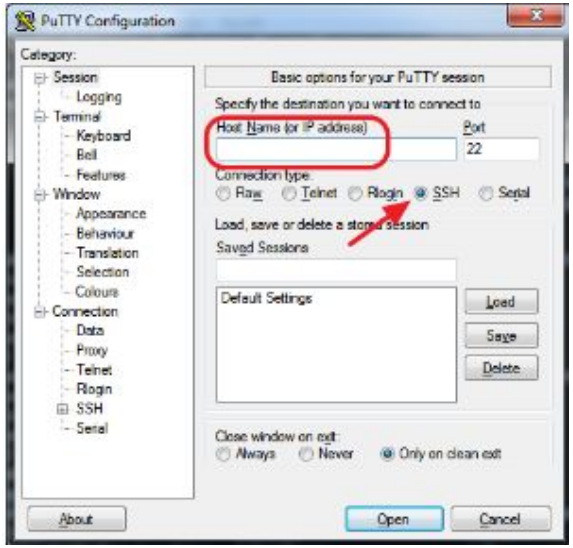
Programa que se ejecuta en nuestro ordenador para iniciar la conexión, indicando el nombre del servidor o su dirección IP.

Desde la terminal

```
$ ssh usuario@ip_servidor
```

Con una aplicación de terceros

PuTTY: la misma conexión, pero de manera gráfica.



Ventana de configuración de PuTTY

<resumen/>

HTTP vs SSH de un vistazo

	HTTP / HTTPS	SSH
¿Para qué sirve?	Intercambiar recursos web entre navegador y servidor	Administrar servidores de forma remota
Puerto	80 (HTTP) / 443 (HTTPS)	22
Cifrado	Solo con HTTPS (TLS)	Siempre cifrado
Herramienta	Navegador, APIs	Terminal o PuTTY
Ejemplo	https://www.ejemplo.com	ssh usuario@ip_servidor

¡Gracias!

¿Preguntas?

```
GET /preguntas HTTP/1.1
HTTP/1.1 200 OK
```